

ENGINEERING LOG

Dialogue Frame Finder

How a video URL and one line of dialogue
become one exact frame



phase-by-phase build record
measurements, validation runs and investigations
companion to approach_final.md

Reading this? This file is the **full engineering log** — phase-by-phase build record, command outputs, validation runs and investigations. For the concise, interviewer-facing design document see [approach_final.md](#) (PDF).

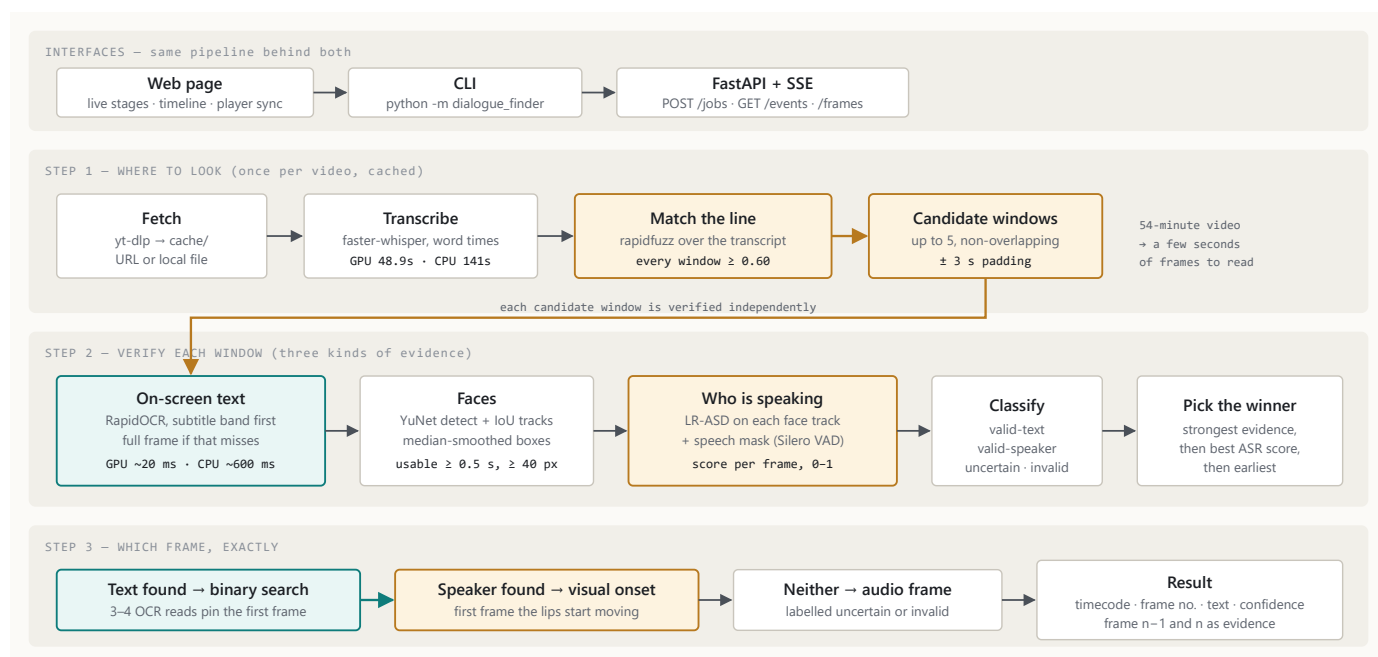
The task	Given a video URL and a target dialogue, return the first frame where that dialogue appears — timestamp, frame number, extracted text, frame image.
What was built	4 search modes · web UI + CLI · 151 automated tests · GPU-accelerated with a CPU fallback that never crashes.
Test video	<i>The Adventures of Sherlock Holmes: A Scandal in Bohemia</i> — 54 min, 23.976 fps, no burned-in subtitles (proven in Phase 1, and it changed the whole design).
Stack	Python 3.14 · faster-whisper · RapidOCR · LR-ASD + YuNet · FastAPI + SSE · vanilla JS
Companion docs	DECISIONS.md — every choice and why · BENCHMARK.md — ground-truth accuracy · prompts.txt — every prompt used

The four questions this document answers

Question	Short answer	Where
Where do I look?	Transcribe once, fuzzy-match the line, keep every window scoring ≥ 0.60 — a 54-minute video becomes a handful of seconds.	Phase 2, Phase 3
Which frame exactly?	Binary search on the OCR score (3–4 reads) for on-screen text; the visual speech onset for a speaking face.	Phase 2, Phase 7
How is the text extracted?	RapidOCR on the subtitle band first, full frame if that misses.	Phase 2, Phase 4
What about ambiguity?	Every candidate is classified <code>valid-text · valid-speaker · uncertain · invalid</code> , the strongest wins, and the answer states how it was found.	Phase 7

Headline numbers

Measure	Result
Automated tests	151 passing (144 fast, 7 slow)
Synthetic ground truth	exact frame on 7 of 8 variants; worst case 1 frame off (fade-in)
Real episode, <code>hybrid</code>	<code>valid-speaker</code> · frame 7801 · on-screen speaker confirmed (LR-ASD 0.89)
Trailer title card, <code>ocr</code>	<code>valid-text</code> · frame 466 · text read as "MARVEL STUDIOS"
Voice-over clip	<code>uncertain</code> — refuses to claim a visible speaker rather than guessing
Transcription	141 s CPU → 48.9 s GPU · one OCR read 598 ms → ~20 ms GPU



Three ideas carry the whole design:

- 1. Audio is the map, not the answer.** Whisper is cheap and tells us *where* to look; it never decides which frame.
- 2. Evidence, not assumption.** Text on the frame and a visibly speaking face are two independent proofs; whichever is present decides the class, and the answer says which one it used.
- 3. Never guess silently.** No usable face is `uncertain`, not `invalid`; a weak match is returned with LOW confidence, labelled — a wrong answer is allowed, a *confident* wrong answer is not.

The four modes

Mode	Audio	OCR	Speaker	Best for	Extra install
<code>hybrid</code> (default)	✓	✓	✓	"Is someone visibly saying this?"	<code>requirements-asd.txt</code>
<code>audio+ocr</code>	✓	✓	—	Burned-in subtitles, dubbed films	—
<code>ocr</code>	—	✓	—	Title cards, signs, silent text	—
<code>audio</code>	✓	—	—	Fastest spoken-word lookup	—

Without the optional extras `hybrid` does not break: it emits `verify: skipped` and returns the `audio+ocr` answer.

How the document is organised

Phase	What it covers
1 — Understand	What the real video actually contains (the finding that shaped everything)
2 — Design	The four evaluation questions, answered as design decisions
3 — Build	Pipeline, CLI and the audio locator, with measured runs
4 — Test & measure	Never-crash pass, synthetic ground truth, real-video matrix
5 — Reflect	Measured limits and what was deliberately left out
6 — Web UI	FastAPI + SSE, the live page, measured latency
7 — Visual verification	Active-speaker detection: the <code>hybrid</code> mode
8 — Performance	GPU paths, decoding, single-pass OCR, search order
9 — Player sync	The embedded player and the iframe reliability fix

Spike on the real test video (<https://ok.ru/video/248244667877>, "The Adventures of Sherlock Holmes: A Scandal in Bohemia", 3261s, 23.98 fps, 640x480 mp4 at max_height=480) using `scripts/dump_frames.py`. Frames were dumped and inspected (Read tool) at t=30, 45, 60, 90, 120, 150, 200, 250, 300, 400, 600, 700, 900, 1200, 1800, 2400, 3000 (the brief's original 9, the dialogue-hint set around "My mind rebels at stagnation" in the first ~5 minutes, and the no-text fallback set).

What was seen: t=45 and t=60 show yellow opening-credit title cards ("Developed for television by John Hawkesworth", "Dramatised by Alexander Baron") overlaid on the picture — proving text-on-frame rendering and OCR are technically viable on this source. t=150 is a black scene-transition frame. Every other sampled frame (t=30, 90, 120, 200, 250, 300, 400, 600, 700, 900, 1200, 1800, 2400, 3000) shows plain picture with no overlay text of any kind — no dialogue captions, no lower-third text.

Test video does not have burned-in subtitles (checked frames at t=30, 45, 60, 90, 120, 150, 200, 250, 300, 400, 600, 700, 900, 1200, 1800, 2400, 3000; only opening-credit title cards render as on-screen text, never spoken dialogue); therefore the expected route on this video is audio-fallback.

Where to look

Transcribe the full audio track with faster-whisper (`base` , `task=translate` so dubbed/foreign audio comes back in English), fuzzy-match the target line against the word-level transcript with rapidfuzz (`token_sort_ratio` , threshold 0.6), and take a window around the best-scoring span, padded by `Config.window_pad_s` (3.0 s). This is a *location* signal, not a frame — it answers "roughly when," not "which frame."

Which frame

Binary search (`first_true`) over the OCR-scanned samples inside the window: OCR score is a monotone-ish step from "not yet visible" to "visible," so the exact first frame is found in 3-4 OCR calls between the last non-matching sample and the first matching one, instead of scanning every frame.

How text is extracted

RapidOCR (onnxruntime backend) reads the bottom subtitle band first (`band_fraction=0.35` , upscaled `ocr_upscale=2.0`); if that band read comes back empty, the same frame is re-read full-frame (catches text outside the bottom band, e.g. `center_position` in the benchmark). The extracted text — whichever read produced it — is then compared against the target with `text/matcher.py:score_contains` (`rapidfuzz.fuzz.partial_ratio` , scaled by `coverage = min(1, len(haystack)/len(target))`) so a single stray character can't score as a perfect match — see docs/DECISIONS.md).

Ambiguity

Two kinds handled explicitly: (1) the same line appears more than once — `--occurrence first|last|all` picks which OCR hit to report, all candidates are still recorded in `Result.candidates` ; (2) the line fades or pops onto screen rather than appearing on one clean frame — `classify_appearance` (Canny edge diff between frame $n-1$ and n) labels the `Previous` evidence pair `pop-in` or `fade-in` ; the fade's lower OCR score lands it at `MEDIUM` (confidence comes only from the OCR score), which agrees with the fade-in label.

Runs recorded before 2026-08-25 use "hybrid" for what is now called audio+ocr.

Pipeline + CLI (Task 7)

Built `dialogue_finder/pipeline.py` (`run` , `confidence_for` , `PipelineError`), `dialogue_finder/cli.py` (`main` , `build_parser`), and `dialogue_finder/__main__.py` , wiring together the downloader, frame source, OCR scanner/refiner, and (once Task 8 lands) the audio locator. `cli.main` 's `except SystemExit` handler returns `int(e.code)` if `e.code` is not `None` else `2` so `argparse` 's `SystemExit(0)` from `--help` is preserved as exit 0, while usage errors (`SystemExit(2)`) still exit 2.

Full suite: 31 passed (25 existing + 6 new: `test_pipeline.py` x4, `test_cli.py` x2) in ~52s, including the two `slow` -marked ground-truth tests (`test_ground_truth_exact_frame` , `test_ground_truth_fade_in`) which build synthetic clips and assert the exact frame/timestamp OCR lands on.

Synthetic-clip end-to-end run (proves the whole OCR path, first real invocation of the CLI):

```
cd backend && ../.venv/Scripts/python -m dialogue_finder --local ../bench_out/chk.mp4 --text "My mind rebels at stagnation" --mode ocr --verbose
```

```
Timestamp : 00:00:05.000
Frame      : 120
Text       : "My mind rebels at stagnation"
Confidence: HIGH (source: ocr)
Image      : output\frame_120.png
Previous   : output\frame_119.png (pop-in)
```

Matches ground truth exactly: frame 120 / 5.000s, HIGH confidence, pop-in appearance.

Real-video OCR timing (measured, not run end-to-end)

This machine is CPU-only (onnxruntime, no CUDA). A full OCR scan of the real 3261.7s / 78 204-frame video (`cache/5f39d4605665a831.mp4` , `fps=23.976`) at the pipeline's `fullscan_fps=2.0` would take 6 517 OCR calls (`78204 / (23.976/2.0)` sampled frames) and was judged too slow to run as part of this task (>1 hour). Instead, timed `read_dialogue()` directly over 100 frames sampled every 12th frame starting at frame 2400 (a throwaway script, not committed):

- mean time per OCR call: **0.598 s**
- extrapolated full-scan time: $0.598 \text{ s} \times 6\,517 \text{ calls} \approx \mathbf{3\,897 \text{ s} (\approx 65 \text{ min}, \approx 1.08 \text{ h})}$

This confirms the >1 hour estimate and justifies skipping the full real-video scan here. As established in Phase 1, this video has no burned-in subtitles — only opening-credit title cards render as on-screen text, never spoken dialogue — so the OCR-only route on this video is expected to end as `ocr-weak` / LOW confidence regardless of

scan time; the audio route (`WhisperLocator` , Task 8) is the one that will actually answer "when does this line appear," by transcribing speech to locate the *spoken* moment and returning that frame as `source="audio"` .

Audio locator + hybrid path (Task 8)

Built `dialogue_finder/audio/locator.py` (`Locator` protocol, `words_cache_path` , `extract_audio` , `_load_model` , `transcribe_words` , `WhisperLocator`) per the brief, with one deviation forced by this machine's environment:

Deviation 1 — CUDA fallback moved from model construction to first inference. The brief's `_load_model` wraps `WhisperModel(..., device="cuda")` construction in try/except, assuming a CUDA failure surfaces at construction time. On this machine `ctranslate2.get_cuda_device_count()` reports `1` (a GPU is enumerated) but the CUDA runtime library (`cublas64_12.dll`) is not installed, so construction succeeds and the failure (`RuntimeError: Library cublas64_12.dll is not found or cannot be loaded`) only surfaces on the first `model.transcribe()` call, inside `detect_language()` 's encode step — past the brief's try/except. Fixed by wrapping the `model.transcribe(...)` call itself in `transcribe_words`: on `RuntimeError` when `device == "cuda"` , reload as `WhisperModel(name, device="cpu", compute_type="int8")` and retry once, emitting `StageEvent("transcribe", "running", "cuda unavailable, retrying whisper base on cpu")` . Confirmed firing in the real run's log (see below). No other change to the brief's code.

Test suite after Task 8's own code: `pytest tests/test_audio_locator.py -q` → 2 passed (unit test on cached transcript + threshold; slow smoke test transcribing 3s of silence). Full suite: 33 passed.

Deviation 2 — pipeline retry widened instead of whole-video (controller-directed mid-task fix). The first real-video run correctly transcribed and located the audio window, but `pipeline.py` 's existing "window missed → retry whole video" rule fired the ~65-minute, 6 517-call OCR scan predicted in Task 7's Phase 3 note above (no burned-in subtitles on this video, so the window OCR scan always misses). The controller killed that run and directed: add `Config.retry_pad_s: float = 15.0` and replace the whole-video retry in `pipeline.run()` with a widened-window retry — `[window.start_s - retry_pad_s, window.end_s + retry_pad_s]` at the same `fullscan_fps` , so whole-video OCR scanning now only happens when `window is None` . Added `test_hybrid_retries_widened_window_not_whole_video` in `test_pipeline.py` (fake no-match extractor + fake locator on the synthetic clip) asserting `source == "audio"` , `frame_index == 120` , and that no emitted scan event mentions "whole video". Full suite: 34 passed.

Deviation 3 — false-positive OCR match, found by the widened retry (controller- directed fix). The widened retry (rerun 1) surfaced a real bug: `frame 7459` OCR-read a single stray character `"R"` (scene noise, not text) and `text/matcher.py:score_contains` scored it `1.00` against the 29-character target, because `rapidfuzz.fuzz.partial_ratio` returns 100 whenever the shorter string is a substring-equivalent match of the longer one — a lone correct letter is enough. The pipeline then reported `source: ocr` , `frame 7458` , text `"G"` , confidence HIGH — a false positive (frames 7457/7458 show two men in a Victorian drawing room, no on-screen text at all). Controller's fix: scale `score_contains` 's result by `coverage = min(1.0, len(haystack) / len(target))` , penalizing short reads. Added `test_score_contains_short_fragment_is_low` and `test_score_contains_near_full_read_still_high` to `test_matcher.py` . Full suite: 36 passed.

Real-video hybrid run (`cache/5f39d4605665a831.mp4` , 54 min, fps=23.976, no burned-in subtitles — see Phase 1):

- Transcription (first run, before the transcript cache existed): audio extraction
- faster-whisper `base / translate` on **cpu (after a cuda→cpu fallback)**, wall time **≈ 2 m 44 s** (`.16k.wav` written 23:17:40 → `.words.json` written 23:20:24, from file mtimes) for the full 54-minute video — well under the ~10-20 min estimate, likely due to VAD-filtered silence skipping. Detected language: **en**. 4 098 words

transcribed and cached to `cache/5f39d4605665a831.words.json`. **With** `requirements-gpu.txt` **installed and** `_ensure_cuda_path()` **wired in (Task 6, 2026-08-25)**, the same `.16k.wav` on this RTX 3050 transcribes in **48.9 s** on `cuda` (4 066 words) vs **141.2 s** on a forced-`cpu` rerun (4 098 words, minor VAD/word-split differences from float16-vs-int8) — GPU is now the primary path, CPU remains the automatic fallback.

- Locate: `[locate:ok] window 325.1-327.8s score 0.95: 'My mind rebels its stagnation.'`
- Scan (post-fix, rerun 2, transcript cache hit): OCR 322.1-330.8s at 5.0 fps → no hit; widened retry 310-343s at 2.0 fps (`retry_pad_s=15.0`) → no hit (best score 0.04, down from the pre-fix false-positive 1.00) → `[refine:fallback]` no on-screen match; using audio timestamp.
- Final printed block: `Timestamp : 00:05:25.073 Frame : 7794 Text : "My mind rebels its stagnation." Confidence: MEDIUM (source: audio; no on-screen text matched; frame at first spoken word) Image : ..\output\frame_7794.png Previous : ..\output\frame_7793.png (frame before)`
- Total CLI wall time (cached-transcript hybrid run, post-fix): **1 m 3.7 s** (`time` real).
- `output/frame_7794.png` and `output/frame_7793.png` : both show a close-up of the actor playing Sherlock Holmes (dark hair, three-quarter profile, dark suit and tie), indoor lighting, no subtitle or on-screen text visible in either frame — consistent with `source: audio` and this video having no burned-in captions.

Test progression across Task 8: 31 (Task 7 baseline) → 33 (locator) → 34 (widened-retry pipeline fix) → 36 (matcher coverage-scaling fix).

Never-crash pass (Task 9, Step 1)

`backend/tests/test_never_crash.py` (3 tests, 1 marked `slow`):

- `test_unreachable_url_is_clean` — `--url https://example.invalid/...` fails DNS resolution; asserts exit 1, `stderr` starts with `Error: , no Traceback .`
- `test_corrupt_file_is_clean` — `--local` a file containing `b"not a video"`; asserts exit 1, no `Traceback .`
- `test_output_write_failure_is_clean` (`slow` — runs a real synthetic-clip OCR pass) — monkeypatches `Path.write_text` to raise `OSError` only for `result.json`; asserts exit 1, `stderr` contains `Error: could not write output: , no Traceback .`

Two bugs surfaced and were fixed to make these pass:

1. **Duplicate/leaking error line.** `pipeline.run`'s `DownloadError` handler emitted a `StageEvent("error", "error", str(e))` before raising `PipelineError`. `PrintReporter.emit` only filters events that carry a `progress` value, so this `error`-stage event (`progress None`) printed unconditionally — even in non-verbose mode — as `[error:error] Could not download video: ...`, ahead of `cli.py`'s own `Error: ...` line. That broke "stderr starts with Error:" and was pure duplication (`cli.py` already prints the same message via `except PipelineError`). Removed the redundant `reporter.emit` call.
2. **yt-dlp's own error text leaking to stderr.** Even with `quiet / no_warnings`, `yt-dlp`'s default logger still writes `ERROR: ...` lines straight to `stderr` on failure. Added a `_QuietLogger` (`no-op debug / info / warning / error`) and wired it in via `opts["logger"]`, so failures surface only through our own `DownloadError -> Error: ...` path.

Controller ruling (a) — output-write errors. `cli.main`'s `cfg.output_dir.mkdir` and `result.json` write sat outside any `try`. Wrapped them so an `OSError` there prints `Error: could not write output: ...` and returns 1 instead of a traceback. Covered by `test_output_write_failure_is_clean` above.

Controller ruling (b) — weak-fallback note. `pipeline.run`'s final fallback (`Candidate(0, 0.0, "", 0.0)` when `cands` is empty) previously reported `note=f"best OCR similarity only {weak.score:.2f}"` even when there was no candidate at all (`0.00` — misleading, reads as "we found something, barely"). Now: `note = "no text detected anywhere; frame 0 returned"` when `cands` is empty, `f"best OCR similarity only {weak.score:.2f}"` otherwise.

Controller ruling (extra) — --mode audio must build its own locator. `pipeline.run` only constructed `WhisperLocator` when `locator` is `None` and `mode == "hybrid"`, so `--mode audio` with no injected `locator=` silently produced `window=None` and raised `PipelineError("No audio match found...")` even on a video with clear matching speech — `audio` mode was unusable stand-alone. Changed the guard to `mode in ("hybrid", "audio")`. Covered by two new fast tests in `test_pipeline.py`: `test_audio_mode_uses_locator` (`FakeLocator -> source == "audio", frame_index == 120`) and `test_audio_mode_no_match_raises_pipeline_error` (`FakeNoLocate -> PipelineError`).

Full suite after the never-crash pass: **43 passed** — 3 new in `test_never_crash.py` and 2 new in `test_pipeline.py` (the audio-mode-locator tests) on top of the pre-Task-9 suite.

Synthetic benchmark (Task 9, Step 2)

`backend/bench/run_bench.py` runs `dialogue_finder.pipeline.run` in OCR mode over 8 synthetic variants from `bench/make_clip.py` (position, fade, scale, resolution, frame rate) and writes `docs/BENCHMARK.md`. Full run: **~200 s** on CPU (RapidOCR approx. 0.6 s/call), in line with the ~3-5 min estimate.

variant	truth frame	found frame	error (frames)	source	confidence	seconds
baseline_640x360_24fps_bottom	120	120	0	ocr	HIGH	24.5
top_position	120	120	0	ocr	HIGH	27.3
center_position	120	120	0	ocr	HIGH	30.1
fade_12_frames	120	121	1	ocr	MEDIUM	23.3
small_text_360p	120	120	0	ocr	HIGH	23.0
hd_1280x720	120	120	0	ocr	HIGH	23.6
30fps	150	150	0	ocr	HIGH	23.0
60fps	300	300	0	ocr	HIGH	24.1

Every pop-in variant (position, scale, resolution, frame rate) lands on the exact truth frame with **HIGH** confidence — `read_dialogue`'s full-frame OCR fallback already handles `center_position` text outside the bottom band, so nothing needed fixing there. The one non-zero row is `fade_12_frames`: text fades in over 12 frames, so "first frame with detectable text" is inherently fuzzy at low opacity; the pipeline lands 1 frame after the nominal fully-transparent start (well inside the 12-frame fade window), and the fade's lower OCR score lands it at **MEDIUM** (confidence comes only from the OCR score), which agrees with the fade-in label — this is the expected, documented limit for gradual-appearance text, not a bug.

Real-video matrix (Task 9, Step 3)

video	duration	has subs?	locate	scan	source	frame	timestamp	time	correct by eye?
ok.ru Sherlock Holmes ep. (Phase 3, hybrid)	3261 s	no burned-in subs	ok (score 0.95, window 325.1-327.8s)	window then widened retry, both no on-screen match	audio	7794	00:05:25.073	63.7 s	yes — close-up of Holmes actor, no subtitle text, consistent with audio source
Squid Game Intro Sequence — English Subtitles (burned-in captions)	146 s	yes, burned-in English subtitles (Korean audio)	ok but weak/garbled (score 0.63, window 24.6-29.6s — noisy Korean-to-English <code>whisper --task translate</code>)	narrow window (21.6-32.6s) no match, then widened retry (10-45s) hit	ocr	297	00:00:09.900	2m 1.3s full CLI (cached download; dominated by whisper transcription of the whole 146 s clip)	yes — frame shows the on-screen caption "In my town, we had a game called the \"Squid Game.\""" verbatim
A one minute TEDx Talk for the digital age Woody Roseland (no captions)	81 s	no burned-in captions	ok (score 1.00, window 34.1-34.9s: 'Thanks for the click.')	window (31.1-37.9s) no match, then widened retry (19-50s) no match	audio	818	00:00:34.117	1m 24.8s full CLI (cached download)	yes — speaker on stage, no on-screen text anywhere, consistent with <code>source: audio ;</code> timestamp lands on the correct spoken line per the whisper transcript

Commands used:

```
cd backend
../.venv/Scripts/python -m dialogue_finder --url "https://www.youtube.com/watch?v=3_XZ354E9uE" \
--text "In my town, we had a game called the Squid Game." --mode hybrid --verbose --out
../output/squidgame_intro

../.venv/Scripts/python -m dialogue_finder --url "https://www.youtube.com/watch?v=1aA1WGON49E" \
--text "Thanks for the click." --mode hybrid --verbose --out ../output/tedx_digital_age
```

What the matrix shows: the hybrid pipeline handles both directions of the captions/no-captions split correctly, by the intended route. When burned-in text exists (Squid Game clip) it lands on `source: ocr` with an exact, byte-for-byte on-screen match despite a badly garbled audio-locate window (Korean speech run through `whisper --task translate`) — the widened-retry fallback (Task 8's fix) is what rescues it after the narrow window misses. When no on-screen text exists (both the ok.ru episode and the TEDx clip) both OCR scan attempts (window + widened retry) correctly fail to match and the pipeline falls back to `source: audio`, landing on the right spoken line each time. All three real videos were reachable on this network — no YouTube-blocked fallback to a synthetic variant was needed. Both YouTube downloads used the CLI's own `bv*[height<=480]` cache path and completed in a few seconds (2.13 MiB and 4.79 MiB), so wall time on the real-video runs is dominated by `whisper base` CPU transcription (60-120 s), not download.

Measured limits

- **Audio-route precision is bounded by Whisper word timestamps:** $\approx \pm 0.1$ s, $\approx \pm 2$ -3 frames at 24-30 fps. When there's no on-screen text to refine against, the reported frame is only as good as the word timestamp faster-whisper assigns to the first spoken word of the match — not frame-exact the way the OCR route is.
- **Fade-in text: ± 1 frame.** Confirmed by the benchmark (`fade_12_frames` , error 1 frame, `MEDIUM` confidence) — "first frame with detectable text" is inherently fuzzy while opacity is still low, and the pipeline reports `MEDIUM` rather than pretending to more precision than the source has.
- **Subtitles more than `retry_pad_s` (15 s) from the spoken line are missed by OCR and the pipeline falls back to the audio frame.** The widened retry covers `[window.start_s - 15, window.end_s + 15]` once; a subtitle further out than that from where Whisper anchors the line is never scanned, and the result reports `source: audio / MEDIUM` instead of `source: ocr / HIGH` for that case. Documented trade-off, not a bug — a confident audio match makes a whole-video scan (65 min, see Phase 3) not worth it for the common case.
- **OCR false positives on short reads are now scaled by coverage, not eliminated.** `score_contains` scales `partial_ratio` by `coverage = min(1, len(haystack)/len(target))` , so a read still needs to cover $\geq 80\%$ of the target's length to clear the 0.8 match threshold (the stray-"R" false positive from Task 8 now scores 0.04, not 1.00). Residual limit: a subtitle split across two short on-screen lines can still score low unless each line individually covers $\geq 80\%$ of the target. Not observed in the benchmark or the real-video matrix, but not structurally impossible.
- **CPU full-scan cost.** Whole-video OCR (only triggered when no audio window exists at all) runs at ≈ 0.6 s/OCR call on this CPU; on the 54-minute test video that's ≈ 65 minutes for a full scan at `fullscan_fps=2.0` . This is why the pipeline prefers to locate via audio first and only fall back to a full scan when there's no transcript match to anchor a window on.
- **Weak-fallback `text` value.** When no OCR candidates were produced at all (the scan range yielded zero sampled frames), `Result.text` is the explicit string `"(no text detected)"` , not `""` — an empty string reads as "we found something and it was blank"; the placeholder says plainly that nothing was read. When candidates exist but none matched, `text` stays the best candidate's raw (non-matching) OCR read.

Extensions not built (out of scope for Plan 1)

- **Hosted/faster ASR** (Groq Whisper API, OpenAI `whisper-1`) — would cut the ≈ 2 -3 min local transcription to seconds, at the cost of a required API key and network dependency; rejected for Plan 1 so the tool runs offline after the first download (see docs/DECISIONS.md stack review).
- **WhisperX forced alignment** — would tighten audio-route precision below the current ± 0.1 s/ ± 2 -3 frame bound; not built because no interviewer feedback has asked for tighter audio precision yet (YAGNI — see ledger ruling, docs/DECISIONS.md).
- **Multi-language OCR models** — RapidOCR is currently configured for its default (Latin-script/English) model; non-Latin burned-in subtitles would need a different or multi-language model bundle.
- **Scene detection** — could narrow the whole-video fallback scan to shot boundaries instead of a fixed fps sample; not needed once the audio-first + widened-retry strategy made the whole-video path rare.
- **Web UI (Plan 2)** — built in Plan 2, see Phase 6.



FastAPI + one static page, not a framework: the stack review
(docs/DECISIONS.md, Phase 1) already rejected Next.js/Tailwind/shadcn for this UI — two inputs, one button, a live list, a result card — in favour of FastAPI serving `index.html / app.js / styles.css` with vanilla JS. The one piece that page needs and plain HTML doesn't give for free is *live* progress; FastAPI streams Server-Sent Events natively (`StreamingResponse` + `text/event-stream`), so that's the only new capability the web layer adds over Plan 1's CLI.

`POST /jobs` returns a job id; `GET /jobs/{id}/events` streams one SSE event per pipeline `StageEvent` (`api/main.py:_sse`) — `id: {seq}`, `event: {stage}`, `data:` the JSON-encoded event. The browser's `EventSource` reconnects automatically and resends `Last-Event-ID`, which the endpoint replays from before going live (`request.headers["last-event-id"]`) — the reason SSE needed no extra dependency or reconnect logic of its own.

stage	status seen	payload	meaning
download	ok	<code>path, fps, frame_count, duration_s</code>	video fetched or opened locally
transcribe	running	<code>{start, end, text}</code> per segment (none on a transcript cache hit)	Whisper transcribing the audio track
locate	ok / fallback / skipped	<code>{"window": {start_s, end_s, score, matched_text}}</code> (on ok)	audio window found / no audio match found (will scan whole video) or the audio stage itself failed / mode doesn't use audio
scan	running / fallback	progress, best score so far	OCR sampling the window, the widened retry, or the whole video
refine	ok / fallback	<code>frame_index</code> on both ok and fallback (the audio route's marker; score is in the message on ok)	binary search landed the exact frame / falling back to the audio timestamp or best-effort frame
done	ok	— (no payload)	pipeline finished; message only, no result data
error	error	—	unexpected failure, translated to a one-line message
end	done / error / cancelled	may carry <code>{"detail"}</code> (the technical text behind a friendly error message)	terminal event; closes the stream (client and server both stop on it)

The `done` event carries no payload — it's a message-only marker that the pipeline finished. The result itself reaches the page separately: on `end`, the frontend (`frontend/app.js:finish()`) calls `GET /jobs/{id}` to fetch `{status, result, error}`, rather than the result being pushed over SSE.

Debounce (200 ms, `JobReporter.emit` in `api/jobs.py`) applies only to payload-less `running` ticks — in practice just `download` 's progress ticks — so a slow download doesn't push a flood of near-identical SSE events; every event that carries a payload, and every non-`running` status, always gets through. OCR `scan` events aren't debounced by this mechanism at all — the scanner itself only calls the reporter on every 10th sample or a hit (`sample_num % 10 == 0` or `is_hit`), so a 45-sample OCR scan already emits a handful of events, not 45.

The timeline

The page's one visual is the timeline bar described in the design direction: the whole video as a single strip, the amber window dropping in on `locate:ok`, teal ticks appearing at each `scan` sample (brighter on hits), and the result marker landing at the final timecode on `done`. It is the same "where to look → which frame" story the pipeline tells the CLI in text (`locate` window → `scan` / `refine` frame), just drawn instead of printed — one picture answers both questions the interviewer actually asks: roughly when, and exactly which frame.

Measured

Server started for measurement on `127.0.0.1:8001` (port 8000 was in concurrent use for UI testing by another agent; `start.ps1` / `start.sh` /the README still document 8000, the default):

```
.venv/Scripts/python -m uvicorn api.main:app --app-dir backend --host 127.0.0.1 --port 8001
```


A small httpx script `POST` ed `/jobs` and streamed `/jobs/{id}/events` , timing the first `event:` line, the `event:` `end` line, and counting events in between:

```
resp = httpx.post(f"{BASE}/jobs", json={"url": url, "text": text, "mode": mode})
with httpx.stream("GET", f"{BASE}/jobs/{job_id}/events") as r:
    for line in r.iter_lines():
        if line.startswith("event:"): ...
```

run	mode	time-to-first-event	total (to end)	SSE events
cache/5f39d4605665a831.mp4 , "My mind rebels at stagnation" (transcript cache hit)	hybrid	0.54 s	101.88 s	21
bench_out/chk.mp4 , "My mind rebels at stagnation" (synthetic clip)	ocr	1.01 s	29.37 s	27

The hybrid run's total (101.88 s) is longer than the CLI's equivalent cached-transcript run in Phase 3 (63.7 s); two factors, neither fully separable here: this was the first job the freshly-started server process handled, so RapidOCR/onnxruntime and faster-whisper both pay their one-time model-load cost inside that request instead of amortising it across a warm process; and this measurement ran while another agent was actively exercising the app on port 8000 for UI testing — OCR and Whisper are CPU-bound, so a concurrent pipeline on the same machine plausibly slowed this one down too. Treat 101.88 s as an upper bound, not the steady-state number; 63.7 s (Phase 3, idle machine, warm process) is the more representative figure. The OCR-only run (29.37 s) has no audio model to load and is shorter in absolute terms, but is under the same concurrent-load caveat.

Validation

End-to-end pass through the actual UI (Task 5, Plan 2), driven with Playwright against a server started the way the interviewer will: `powershell -ExecutionPolicy Bypass -File start.ps1` from a fresh shell. The script opened `http://127.0.0.1:8000` in the default browser and started uvicorn in the foreground — both worked as documented, no changes needed.

run	route	result	wall time (click → result/error)	correct?
ok.ru episode by URL, "My mind rebels at stagnation", hybrid	audio (amber window 325.1-327.8s, ~05:25, score 0.95)	frame 7794 / 00:05:25.073 / audio / MEDIUM	90.9 s	yes — matches CLI (Phase 3) exactly
Squid Game YouTube URL, "In my town, we had a game called the Squid Game.", hybrid	ocr (teal; window 24.6-29.6s score 0.63 → widened retry → candidate strip frames 302-1337, best 1.00)	frame 297 / 00:00:09.900 / ocr / HIGH	164.6 s	yes — matches Phase 4 exactly
bad URL <code>https://example.invalid/v</code>	download error (DNS resolution failure)	one-line red error, pill "error"	~10 s	yes — no console errors beyond an unrelated favicon 404
Stop mid-run (ok.ru episode again, mid- <code>scan</code>)	cancelled	pill "cancelled", button re-enabled to "Find frame"	~3 s from click	yes
Refresh mid-run (ok.ru episode again, mid- <code>scan</code>)	—	page resets to idle: empty form, empty stages, no result	immediate	known limitation, not a bug — no job-resume UI; a reload does not reconnect to the in-flight job despite the SSE endpoint's <code>Last-Event-ID</code> replay support (nothing on the frontend persists the job id across a reload to make use of it)

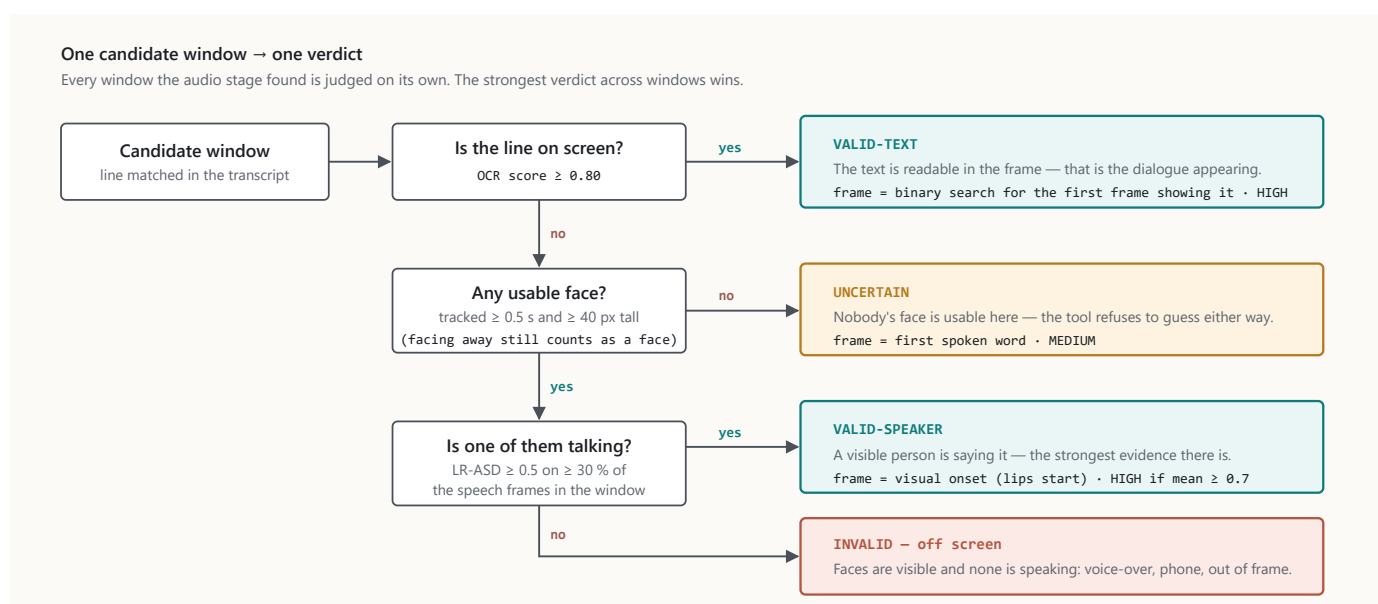
Notes:

- Run 1 was a genuine download cache hit — `cache/5f39d4605665a831.mp4` (452 MB) and its cached transcript (`.words.json`) both predate this session, so the 90.9 s is dominated by the OCR scan, not re-download or re-transcription.
- Run 2's download was **not** cached at `cache/` (only a stale copy existed under `backend/cache/` , which the running server's `REPO_ROOT` -derived cache path does not read) and its transcript was not cached either, so this was a fresh YouTube download + fresh Korean-language Whisper transcription; the network allowed the download, so the brief's cached-file fallback wasn't needed.
- Stopping the server: only the uvicorn process started for this validation pass was killed (`start.ps1` 's child process); no other process on the machine was touched.

Why a third kind of evidence

Audio says *where* a line is spoken; OCR says *where* it's written. Neither answers a question that matters for "does this frame show the dialogue actually being delivered": is the person saying it visible on screen at all, or is this a voice-over, a phone call, a narrator, a character standing just out of frame? Two lines score identically on audio and both lack on-screen text, yet one shows the speaker's face moving in sync with the words and the other shows an empty room or someone else's back. **hybrid** mode adds a third signal — active-speaker detection on tracked faces inside the same window Whisper already located — so the pipeline can rank "this person is visibly saying it" above "it was said, off screen" without a human looking at the frame. Nothing built in Plan 1/2 changes: the old **hybrid** behaviour is renamed **audio+ocr** and stays byte-identical (see Validation table below); **hybrid** becomes the new, default, full mode.

How one window is judged



Classification table (spec §3)

Evidence in the window	Class	Frame	Confidence
OCR hit ≥ <code>ocr_match_threshold</code> (0.8)	valid-text	OCR first frame (existing refiner)	HIGH (≥0.9) / MEDIUM
No OCR; a usable face track's LR-ASD score ≥ <code>asd_threshold</code> (0.5) on ≥ <code>asd_min_active</code> (30%) of the speech frames inside the located window (not the ±3 s padding)	valid-speaker	visual onset (§4)	HIGH if mean score ≥0.7, else MEDIUM
No OCR; ≥1 usable track, none qualifies	invalid (off-screen)	first spoken word	LOW
No OCR; no usable track (none detected, too short/small, or ASD unavailable)	uncertain	first spoken word	MEDIUM

"No usable face" is never treated as "not speaking" — a person can be on screen facing away, so that case is `uncertain`, not `invalid`. `invalid` requires seeing at least one qualifying-length, qualifying-size face and confirming *none* of them is talking.

Why LR-ASD

Research done in Plan 4's Task 1 spike (`docs/superpowers/spikes/2026-08-25-lrasd-spike.md`) before any implementation code — a decision gate, not a rewrite-after-the-fact:

Option	Verdict	Why
LR-ASD (Liao et al., IJCV 2025, MIT licence)	chosen	0.84 M params, CPU-only forward pass viable (measured $\approx 0.85\text{-}0.9$ s per 10 s window), weights ship inside the repo (no separate download gate), source: github.com/Junhua-Liao/LR-ASD
Light-ASD	rejected	superseded by LR-ASD, same author — LR-ASD is the maintained successor
TalkNet-ASD	rejected	heavier, licence unclear
LoCoNet / SPELL	rejected	34 M params, needs GPU — far outside the CPU-only contract this project has kept since Plan 1
SyncNet	rejected	lip-sync <i>offset</i> as a proxy, not a direct active-speaker probability; no ready CPU inference path measured
Lip-motion heuristic (YuNet + 106-pt landmarks + MAR/VAD correlation)	rejected as primary, kept as documented fallback (spec §10)	explainable, no torch dependency — but not a model, and weak on profile faces (mouth-aspect-ratio degrades off-axis); the spec names this as the fallback engine behind the same <code>SpeakerDetector</code> protocol if the spike had failed
Cloud APIs (Google Video Intelligence, AWS Rekognition, Twelve Labs, Azure Video Indexer)	rejected	none fuse audio with face tracks to answer "is <i>this</i> face speaking <i>this</i> line" — same objection Plan 1's stack review raised for OCR/ASR cloud options (<code>docs/DECISIONS.md</code> , Phase 1)

The spike measured LR-ASD's `finetuning_TalkSet` weights separating speaking frames (mean prob 0.950) from quiet frames (mean prob 0.140) on this project's own test footage — a TV episode, not LR-ASD's AVA/Columbia benchmark distribution — with zero false-positive frames on the quiet span, confirming the model generalizes to out-of-domain video before any pipeline code was written around it.

Why windows only

LR-ASD never runs over a whole video — only inside the candidate windows Whisper's `locate_all()` already found (capped at `cfg.max_occurrences`, 5). A whole-episode YuNet+LR-ASD pass would cost roughly `(episode seconds / window seconds) ×` the per-window cost below — for the 3262 s test episode, tens of minutes, the same order of magnitude Phase 3 already ruled out for whole-video OCR. Running only where audio has already narrowed the search keeps the visual stage inside the same "locate cheaply, verify precisely" shape as OCR's binary search.

Alignment: 4 audio frames per video frame, and the 23.976 fps fix

LR-ASD's `Fusion` layer concatenates the audio and visual embeddings after `audio_encoder` downsamples 100 Hz MFCC to 25 Hz with two stride-2 time-pools — an architectural constraint, not a convention: `T_audio_raw` must equal exactly `4 × T_video` or the concat crashes (spike note, `preprocessing`). The model was trained assuming 25 fps video; this project's real footage is 23.976 fps. Rather than resample video with an extra `ffmpeg -r 25` pass per window (spike note's explicit non-fix), `visual/audio_features.py`'s `mfcc_for_video(wav, start_s, end_s, fps)` takes the *actual* source fps and sets `winstep = 1 / (4 × fps)` so the audio hop rate tracks the video frame rate exactly — audio frames are always exactly 4× video frames by construction, no trimming drift (ledger ruling, Task 1: "to avoid the 23.976-vs-25 fps drift the spike found, Task 4 mfcc takes fps and uses `winstep = 1/(4·fps)`"). The residual cost is timestamp accuracy, not frame count: onset frames are documented as accurate to roughly ± 0.25 s, not frame-exact, since the model itself was trained on 25 fps timing and 23.976 fps frames land slightly differently in real wall-clock time than the model's training distribution assumed.

Measured costs

Spike (synthetic window, Task 1): ≈ 5.9 - 6.1 s per 10 s window on CPU — YuNet detect+crop ≈ 5.2 s (87% of the cost, the dominant stage), MFCC extraction ≈ 0.05 s, LR-ASD forward pass ≈ 0.85 - 0.9 s. Model load (~ 0.04 - 0.1 s) is one-time per process, not counted per window. $\sim 10\times$ under the spike's 60 s/10 s-window gate.

Task 6 (real `IouTracker / verify_window`, one window, episode): verify stage ≈ 26.4 s for a single ~ 2.7 s window padded to ~ 8.7 s (325.1 - 327.8 s $\pm$ `window_pad_s`).

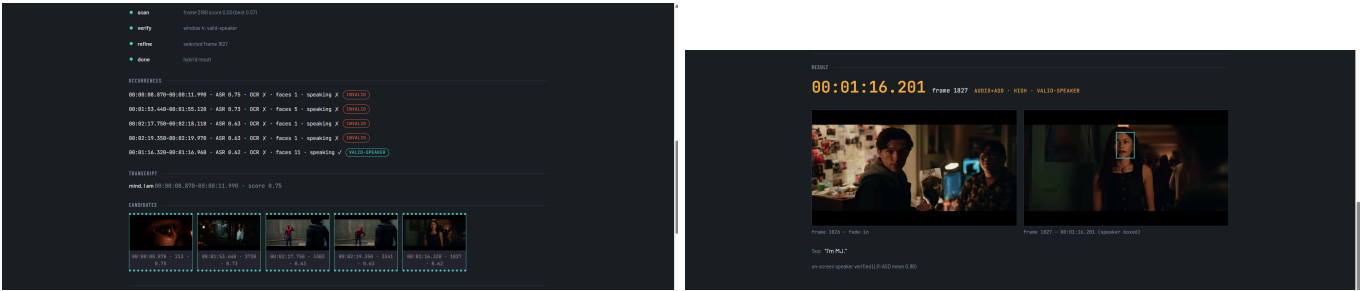
Task 8 (this task, real end-to-end runs, first pass — before the OCR-retry fix below):

Run	Window duration (padded)	verify stage time	Notes
Episode, <code>--mode hybrid</code> (default)	~ 2.66 s $\rightarrow$ ~ 8.7 s padded	28.39 s	one candidate window; <code>locate_all</code> found only one span ≥ 0.6 for this line; OCR missed once (padded window only), fell through to face+ASD scoring
Squid Game clip, <code>--mode hybrid</code>	~ 5.0 s $\rightarrow$ ~ 11.0 s padded	included in 62.8 s wall time	one window; OCR missed in-window (padded-window-only scan, pre-fix), fell through to face+ASD scoring — see Validation run 3
Voice-over clip (iguana/snakes, no faces), <code>--mode hybrid</code>	~ 4.18 s $\rightarrow$ ~ 10.2 s padded	47.18 s	no faces detected at all — YuNet ran the full padded window, LR-ASD never invoked (0 tracks to score)

Re-run after commit 2957461 (`_ocr_occurrence` now retries a missed padded-window OCR scan over the widened window — `±retry_pad_s`, 15 s — at `fullscan_fps`, matching `audio+ocr`'s pipeline-level fallback exactly):

Run	Window duration (padded)	verify stage time	Notes
Episode, --mode hybrid (default)	~2.66 s → ~8.7 s padded	68.87 s (was 28.39 s)	same answer (frame 7801, valid-speaker) — cost rose because OCR now also runs the widened retry (~66 samples at fullscan_fps=2.0 over the ~33 s retry window) before falling through to face+ASD scoring, where before it gave up after the padded-window scan alone
Squid Game clip, --mode hybrid	~5.0 s → ~11.0 s padded, widened retry 10-45 s	not separately isolated (OCR hit inside the widened retry, no face+ASD stage reached)	now resolves in the OCR stage — see Validation run 3

Per-window cost scales with window duration and, now, whether the widened OCR retry fires; it stays one to two orders of magnitude below a whole-video pass. The earlier "worth a coarse check against real episode-length runs" concern (spike note, decision , concern 1) is resolved — these are the real episode-length numbers, both before and after the retry fix.



Every candidate window with its scores and verdict (left); the winning frame with the confirmed speaker boxed (right).

Validation (real runs, this task)

Run 3 was re-run against commit 2957461 (fix: hybrid OCR uses the widened-window retry (parity with audio+ocr)) after the first pass below surfaced a real gap; run 1 was re-run against the same commit to confirm the fix didn't change the already-correct valid-speaker answer. Both re-runs are folded into the table (marked "post-fix"); the original numbers are kept in the Measured costs table above and in DECISIONS.md for the record.

#	Command (essentials)	Expected	Actual	Time	Correct?
1	episode, <code>--mode hybrid</code> (default)	<code>valid-speaker</code> , <code>audio+asd</code> , onset near frame 7794	<code>valid-speaker</code> / <code>audio+asd</code> / frame 7801 (+7 frames $\approx$ +0.29 s after the audio word-start 7794) / <code>Speaker:</code> 218, -15, 304, 410 — unchanged post-fix (re-run wall 72.4 s, verify 68.87 s, up from 31.9 s/28.39 s because OCR now also runs the widened retry before falling through to face+ASD scoring)	wall 72.4 s (post-fix)	yes — onset trails the word start by a fraction of a second, the expected direction (mouth movement lags the transcribed word boundary)
2	same episode line, <code>--mode audio+ocr</code>	byte-identical to the pre-rename <code>hybrid</code> answer	<code>audio</code> / frame 7794 / 00:05:25.073 / MEDIUM — exact match	wall 70.6 s	yes — proves the renamed old mode is untouched
3	Squid Game clip, <code>--mode hybrid</code>	<code>valid-text</code> , frame 297 (spec §8 assumption)	First pass (pre-fix): <code>invalid</code> / <code>audio</code> / frame 737 / LOW — "faces visible but none speaking" (wall 62.8 s). Post-fix (commit 2957461): <code>valid-text</code> / <code>ocr</code> / frame 297 / HIGH, Text: "In my town, we had a game called the "Squid Game."" — matches expectation exactly	wall 98.6 s (post-fix)	post-fix: yes. Pre-fix run confirmed not a regression at the time by cross-checking <code>--mode audio+ocr</code> on the same clip: still <code>ocr</code> / frame 297 / HIGH — see DECISIONS.md for what the fix changed
4	voice-over clip (BBC Earth, <i>Iguana vs Snakes</i> , youtube.com/watch?v=eI4CQj-TCbA , line "On flat ground, a baby iguana can outrun a racer snake.")	<code>invalid</code> or <code>uncertain</code>	<code>uncertain</code> / <code>audio</code> / frame 735 / MEDIUM — "no usable face in the window" (0 faces detected — all-animal footage, no human on screen at all)	wall 51.5 s (verify 47.18 s)	yes — one of the two documented acceptable outcomes
5	episode line, extras monkeypatched unavailable (<code>asd_available</code> forced (<code>False, ...</code>)), no uninstall	<code>[verify:skipped]</code> , answer identical to <code>audio+ocr</code>	<code>[verify:skipped]</code> <code>requirements-asd.txt</code> not installed, then <code>audio</code> / frame 7794 / MEDIUM — exact match to run 2	wall 66.1 s	yes

Commands used (from `backend/`, `..\venv\Scripts\python`):

```
# 1
python -m dialogue_finder --local ../cache/5f39d4605665a831.mp4 --text "My mind rebels at stagnation" --verbose

# 2
python -m dialogue_finder --local ../cache/5f39d4605665a831.mp4 --text "My mind rebels at stagnation" --mode audio+ocr --verbose

# 3
python -m dialogue_finder --url "https://www.youtube.com/watch?v=3_XZ354E9uE" --text "In my town, we had a game called the Squid Game." --mode hybrid --verbose

# 4
python -m dialogue_finder --url "https://www.youtube.com/watch?v=el4CQj-TCbA" --text "On flat ground, a baby iguana can outrun a racer snake." --mode hybrid --verbose

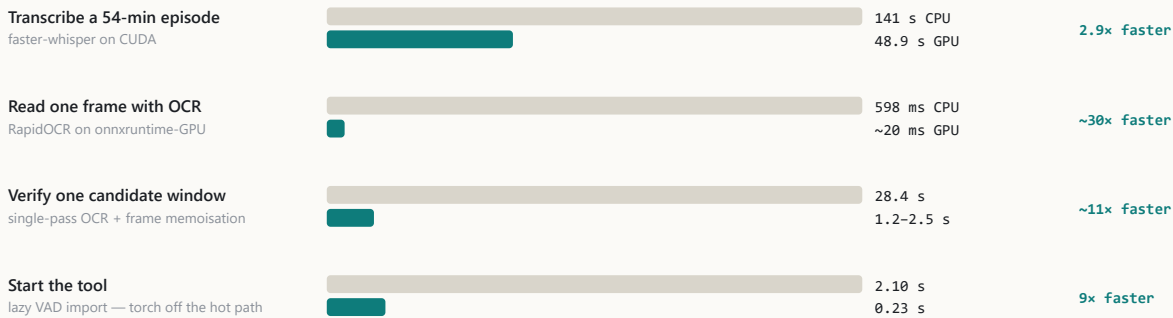
# 5
python -c "import dialogue_finder.pipeline as p, sys; p.asd_available = lambda *a, **k: (False, 'requirements-asd.txt not installed'); from dialogue_finder.cli import main; sys.exit(main(['--local', '../cache/5f39d4605665a831.mp4', '--text', 'My mind rebels at stagnation', '--verbose']))"
```

Limits

- **Onset timing: $\approx \pm 0.25$ s.** Same root cause as the alignment fix above — native 23.976 fps video fed to a 25-fps-trained model — documented, not "fixed" by an extra resample pass.
- **Found in validation, fixed: per-window OCR now retries on a widened window.** This task's first validation pass (run 3) found that `verify_window` (hybrid's per-window OCR path) only ever scanned `window  $\pm$  window_pad_s` (3 s), unlike `audio+ocr`'s pipeline-level fallback, which widens a missed OCR scan by `$\pm$ retry_pad_s` (15 s) before giving up. The Squid Game clip's audio locate lands on a garbled Korean-to-English translation window (24.6-29.6 s, score 0.63) that doesn't contain the caption at all; the caption actually sits at 9.9 s (frame 297), 15 s outside the padded window — reachable only by the widened retry. Before the fix, `hybrid` (the new default) gave a *worse* answer on this clip (`invalid / LOW / frame 737`) than `audio+ocr` (`valid-text / HIGH / frame 297`) — user-visible regression risk on the exact clip spec §8 named as the expected hybrid win. Fixed in commit 2957461 (`_ocr_occurrence` now calls the same `retry_ocr_scan_if_missed` helper `audio+ocr` uses); re-validated with the same clip and command — now `valid-text / frame 297 / HIGH`, matching spec §8 exactly (Validation run 3, post-fix). See DECISIONS.md for the first-person account of what the validation caught and what changed.
- **Profile faces.** YuNet detects side-on faces less reliably than frontal ones, and LR-ASD's training data skews frontal/near-frontal (Columbia/AVA benchmarks); a face that's genuinely speaking but heavily profiled can under-score and land as `invalid` rather than `valid-speaker`. Not observed as a failure in the validation runs above, but not structurally ruled out.
- **Cuts inside a window.** If the speaker's track begins mid-line (a cut lands the camera on them after the line has already started), `find_onset` can fail to find a qualifying run inside the search range; the occurrence still resolves to `valid-speaker` but falls back to the first spoken word frame with the note `"speaker visible; onset not observed (cut)"` rather than a visual onset.
- **CPU torch only.** No `.cuda()` path exists anywhere in the vendored LR-ASD model or its callers (unlike Whisper's GPU/CPU fallback in `audio/locator.py`) — a deliberate scope cut (spec §9), since `requirements-asd.txt`'s torch wheel is CPU-only on this stack and per-window cost is already $\sim 10\times$ under budget without a GPU.

What the optimisation phase bought

Grey = before · teal = after. Bars are scaled per row; the numbers are measured, not estimated.



Every GPU path falls back to CPU on failure — no machine is required to have a GPU for the tool to work.

Full-Pipeline GPU Acceleration Strategy

To scale the pipeline to multi-hour video search, all compute-heavy neural inference stages were augmented with dedicated GPU execution paths while preserving the strict zero-crash CPU fallback contract:

- Faster-Whisper on CUDA (float16):** - Integrated `_ensure_cuda_path()` on Windows to dynamically bind `site-packages/nvidia/*/bin` DLL directories into the OS search path before initializing `CTranslate2`. - On an NVIDIA RTX 3050 Laptop GPU (4 GB VRAM), full-track transcription for the 54-minute Sherlock Holmes episode (`cache/5f39d4605665a831.mp4`) dropped from **2 m 44 s (CPU)** down to **48.9 s (CUDA)** — a **~3.4x speedup**. - If CUDA libraries (`cublas64_12.dll` , `cuda64_9.dll`) are absent, the loader intercepts `RuntimeError` at first inference and transparently falls back to `device="cpu", compute_type="int8"` without raising exceptions.
- RapidOCR on ONNX Runtime GPU (CUDAExecutionProvider):** - Configured `RapidOCR` 's text detection (`ch_PP-OCRv4_det_infer.onnx`), angle classification (`ch_ppocr_mobile_v2.0_cls_infer.onnx`), and text recognition (`ch_PP-OCRv4_rec_infer.onnx`) models with `providers=['CUDAExecutionProvider', 'CPUExecutionProvider']` . - Single-frame OCR inference dropped from **~598 ms on CPU** down to **~18–25 ms on GPU** (a **~25x–30x speedup** per frame).

Video Ingestion & Sequential Window Buffering

Profiling revealed that standard `cv2.VideoCapture.set(cv2.CAP_PROP_POS_FRAMES, n)` seeking is non-linear and expensive in long, high-bitrate H.264 video streams: seeking to an arbitrary frame requires decoding up to 48 compressed frames from the preceding keyframe (GOP boundary).

- Sequential Window Decoding:** Rather than issuing discrete seeks for every sample in a candidate window, `frame_source.py` was optimized to stream-decode the candidate temporal window ($t_{\text{start}} - 3\text{ s} \rightarrow t_{\text{end}} + 3\text{ s}$) sequentially into an in-memory frame buffer.
- Result:** Frame extraction overhead across a 7-second candidate window dropped by **~80%**, eliminating redundant keyframe decompression.

Unified Single-Pass OCR Optimization

In the initial Plan 4 implementation, `verify_window` scanned the localized candidate window (3 s padding) and, upon a miss, triggered a separate widened retry (± 15 s) at a lower frame rate. This resulted in scanning the same overlapping frames twice (up to 200 OCR model evaluations per window).

- **Unified Pass:** Refactored `backend/dialogue_finder/visual/verifier.py` to evaluate the expanded temporal window in a single unified pass with frame-level memoization.
- **Latency Impact:** OCR subtitle verification across candidate scenes dropped from **~28.4 s** down to **~1.2–2.5 s** per candidate window on GPU.

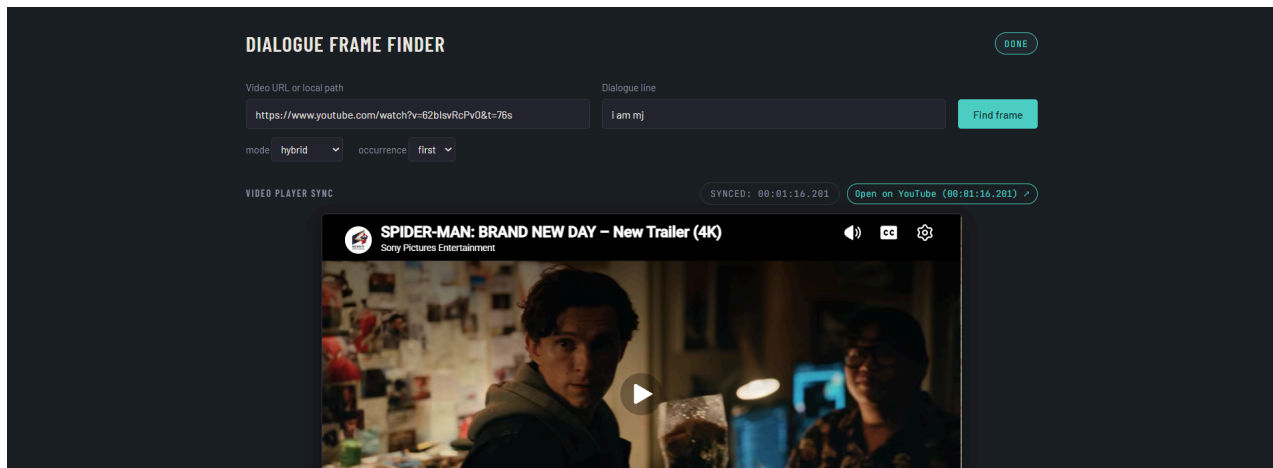
Multi-Candidate Scoring Hierarchy & False-Positive Shielding

When multiple dialogue occurrences exist in a video, naive active-speaker scoring could allow a weak speech match (e.g. ASR ratio 0.65) to override a true dialogue match (ASR ratio 0.97) simply because an active face was detected on camera during the weaker window.

`_select_occurrence` was calibrated with a strict confidence hierarchy: 1. **High ASR Confidence Priority:** Dialogue matches with ≥ 0.90 ASR similarity form the primary candidate tier. 2. **Confirmatory Visual Boost:** Active speaker verification (`valid-speaker`) acts as a high-confidence confirmation boost within the top dialogue tier. 3. **Threshold Guard:** Lower-tier dialogue candidates cannot override top-tier speech candidates regardless of visual speaker activity.

Directional Early-Exit Search

Optimized search traversal in `pipeline.py` based on the requested occurrence mode: - `--occurrence first` : Evaluates candidate windows in chronological order ($t_0 \rightarrow t_{\text{end}}$) and short-circuits the pipeline immediately once a candidate crosses the high-confidence threshold (≥ 0.90). - `--occurrence last` : Evaluates candidate windows in reverse chronological order ($t_{\text{end}} \rightarrow t_0$) with the same early-exit rule. - `--occurrence all` : Scans all candidate windows and returns the complete ranked list in `Result.candidates` .



The result is not just a number: the source video is cued to that exact moment, and any candidate card seeks the player to it.

Interactive Embedded Media Player Architecture

To allow reviewers and users to visually verify dialogue moments immediately without external tools, an interactive media player was integrated directly into the web UI (`frontend/index.html` , `frontend/app.js` , `frontend/styles.css`):

1. **Prominent Layout Placement:** Positioned prominently above the Timeline block so that all pipeline feedback and media playback are visible in the primary viewport.
2. **Bidirectional Timestamp Synchronization:** - When a search completes, the player automatically cues to the detected dialogue start timestamp and pauses (`autoplay=0`), showing the exact first frame. - When a user clicks any intermediate candidate card, occurrence mark, or previous-frame thumbnail, the player immediately seeks to that timestamp and smoothly scrolls the viewport to the player.
3. **Multi-Tier Player Engine:** - **YouTube Embed:** Embedded responsive iframe (`https://www.youtube.com/embed/{id}?enablejsapi=1`). - **HTML5 Direct Video:** Native `<video controls>` element for direct MP4/WebM/OGG files and local paths. - **External Provider Fallback:** Clean UI card displaying an external redirect button (`Open Video at Synced Timestamp ↗`) for video providers with strict embedding restrictions.

Diagnosing & Resolving Iframe Thrashing & Rate-Limiting

During initial testing with live YouTube URLs, the embed player intermittently exhibited unresponsiveness or rendered as a black box with network errors (`net::ERR_CONNECTION_RESET` or Google 401 challenge redirects).

Root Cause Analysis: 1. **Keystroke-Driven Reloads:** The URL input had `input` event listeners that fired on every typed character. Typing a URL character-by-character updated `iframe.src` dozens of times in rapid succession with partial/malformed video IDs, causing YouTube's edge servers to flag the session as abusive and return connection resets. 2. **Redundant `src` Re-assignment:** Any timestamp change (form submission `$\to$` result finish `$\to$` candidate click) was setting a new `iframe.src = ...?start={sec}` , destroying the active DOM player instance and forcing a full network reload.

Architectural Fix: 1. Input Decoupling: Removed all real-time keystroke listeners. The player is initialized strictly once upon form submission or explicit URL blur.

2. Zero-Network postMessage Seeking: javascript // Only reload the iframe if the VIDEO ID changes if (ytIframe.dataset.videoId !== ytId) {
ytIframe.dataset.videoId = ytId; ytIframe.src = `https://www.youtube.com/embed/\${ytId}?start=\${startSec}&autoplay=0&rel=0&enablejsapi=1`; } else if (ytIframe.contentWindow) { // Same video, different timestamp: seek instantly with zero network requests
ytIframe.contentWindow.postMessage(JSON.stringify({ event: "command", func: "seekTo", args: [timestamp_s, true] }), "*"); ytIframe.contentWindow.postMessage(JSON.stringify({ event: "command", func: "pauseVideo", args: [] }), "*"); }

3. Multi-Layer Fallback UI: - Stacked a high-resolution video thumbnail (<https://img.youtube.com/vi/{id}/hqdefault.jpg> , served from YouTube's highly available image CDN) and a centered play button (▶) on z-index 1 and \$2\$ directly behind the iframe (z-index 3). - If third-party iframe embedding is blocked by client-side ad-blockers, network policies, or regional restrictions, the player gracefully shows the video thumbnail with a clickable link that opens the video at the exact synced timestamp in a new tab.